ATTRIBUTION-PATCHING-GUIDED REPLAY REFINEMENT FOR TASK-VECTOR MERGING

Anonymous authors
Paper under double-blind review

ABSTRACT

We study model merging when only a small labeled replay buffer is available for each task. We introduce expert-anchored gradient descent, which can start from any initial merge or directly from the pretrained model. For each task, a first-order loss attribution selects coordinates where moving toward its expert is locally beneficial; updates are scaled by their distance to the expert and clipped to prevent overshoot. This yields an equivalent attribution-patching interpretation. We evaluate the method on multi-head GLUE-8 and the eight- and twenty-task CLIP suites, with every method given the same budget of at most 32 labeled examples. From the pretrained model, the anchored update beats unconstrained descent on every benchmark and every buffer draw, and beats every checkpoint-only merge on the CLIP suites; from merge initializations, it typically improves performance. We further observe that it is good at retaining the pretrained model's ability.

1 INTRODUCTION

Merging fine-tuned variants of a shared pretrained model combines their skills in weight space, without extensive retraining, ensembling, or routing. But merges degrade under task interference. In many practical settings a small *labeled* replay buffer per task is available.

A naive way of using such small buffer is to simply do gradient descent, either on the pretrained model or an initial merge, but ordinary gradient descent can be risky when only a few examples are available.

We propose a refinement that exploits the position of the task experts called *anchored to the task experts*: each coordinate moves toward its expert only where a first-order attribution score predicts that the move reduces the task's replay loss, by an amount proportional to that predicted saving, and never past the expert, from *any* initialization, including the pretrained model itself. The update is provably non-expansive toward the experts' box hull at any learning rate (Proposition 1).

Our contributions are: (i) a replay-refinement method with two equivalent derivations—expert-anchored gradient descent and parameter-space attribution patching—and a divergence-safety guarantee; (ii) a uniform evaluation across three benchmark families (multi-head GLUE-8 and the eight- and twenty-task CLIP suites); (iii) a measurement of what the refinement costs the pretrained model on tasks never merged or refined: the anchored update retains more than every checkpoint-only merge.

2 RELATED WORK AND BACKGROUND

Merging from checkpoints. Model merging combines fine-tuned variants of a shared pretrained model into a single model in weight space. Model soups average compatible checkpoints (Wortsman et al., 2022), and task arithmetic represents fine-tuning as a *task vector* $\tau_t = \theta_t - \theta_0$ and merges as $\theta_0 + \sum_t \lambda_t \tau_t$ (Ilharco et al., 2022). Interference between task vectors—over-counted directions, sign and magnitude conflicts—is the central failure mode, and one family of repairs constructs the merge from the checkpoints alone, differing in the rule that suppresses it: TIES-Merging trims small-magnitude updates and merges only sign-consistent components (Yadav et al., 2023); DARE randomly drops most delta parameters and rescales the rest (Yu et al., 2023), generalized by DELLA (Deep et al., 2024); Model Breadcrumbs removes negligible and outlier perturbations (Davari and Belilovsky, 2023); the magnitude variant of Localize-and-Stitch stitches sparse task-relevant regions back into

the pretrained model (He et al., 2024); and, closest in optimization form to our update, DOGE models merging as adaptive projective gradient descent (APGD), learning a shared modification to the task vectors under a data-free Taylor objective with gradients projected orthogonally to an SVD-derived shared subspace (Wei et al., 2025). "From checkpoints" describes the construction, not the reported numbers: each method carries a few hyperparameters—a scaling coefficient, a trim density—that are conventionally chosen on held-out validation data (Ilharco et al., 2022; Yadav et al., 2023), a cost MergeBench finds to dominate that of merging itself (He et al., 2025); TIES is the exception that also reports a fixed default recipe for the no-validation case.

Merging with unlabeled data. A second family fits the merge on unlabeled inputs or model outputs: Fisher merging weights parameters by a Fisher information estimated from the models' outputs on a data sample (Matena and Raffel, 2022); RegMean solves each linear layer by regression on its input activations (Jin et al., 2023); AdaMerging learns merge coefficients by entropy minimization, in its standard form on the unlabeled evaluation inputs (Yang et al., 2024); and Task Arithmetic in Trust Region (TATR) uses task-loss gradients at the pretrained model to identify low-interference dimensions in which to merge (Sun et al., 2025). Twin-Merging, Task Vector Bases, FusionBench, and MergeBench extend and standardize this space (Lu et al., 2024; Zeng et al., 2025; Tang et al., 2024; He et al., 2025).

Repair with labeled data. When a few labels per task are available, prior work spends them on low-dimensional objects fixed at merge time: LM-Cocktail sets merging weights from each model's loss on a small labeled sample (Xiao et al., 2024); the learned variant of Localize-and-Stitch trains its sparse masks on labeled data (He et al., 2024); Activated Parameter Locating (APL) uses few-shot examples to estimate parameter-partition importance, which determines pruning ratios and model-level merge weights (Kong et al., 2024); task-arithmetic coefficients can be tuned per task on the labels instead of on a validation split (Ilharco et al., 2022); and Fisher weights can be estimated from labeled-loss gradients (Matena and Raffel, 2022). Two further uses come from outside the merging literature and serve here as baselines: refitting only the classifier head on the labeled sample, i.e. linear probing on the frozen merged encoder (Alain and Bengio, 2016; Kumar et al., 2022), and rehearsal—fine-tuning the whole model on a stored buffer of examples (Robins, 1995; Chaudhry et al., 2019)—which in our grids appears as ordinary replay gradient descent. Common to all of these, the labels either shape a handful of coefficients, masks, or head weights, or drive unconstrained descent with nothing tying the model to the experts. The present work sits between the two: it asks whether a small labeled replay buffer can repair an initial merge—or build multi-task ability directly from the pretrained model—while staying close to the known experts, evaluating labeled task gradients at the *evolving* model and taking sequential, coordinate-wise steps toward the experts sized by per-coordinate attribution scores. Section 4 orders all of these baselines by how much label information can reach the reported weights.

Attribution patching. Attribution patching approximates the effect of an intervention by a first-order Taylor expansion, replacing many explicit interventions with gradient-based scores (Syed et al., 2024; Kramar et al., 2024). Most closely related conceptually, APL intervenes by replacing fine-tuned parameter partitions with their pretrained values and approximates their importance by the magnitude of a task-vector–gradient inner product at the pretrained model (Kong et al., 2024). Its static, partition-level scores determine pruning ratios and merge weights. Our method instead evaluates signed, coordinate-level directional derivatives at the evolving model and turns them into sequential, clipped steps from the current initialization toward each task expert. Extended positioning against the benchmark families of the merging literature is given in Appendix C.

3 METHOD

3.1 PROBLEM SETUP

Let $\mathcal{T} = \{1, \ldots, T\}$ be a set of tasks whose experts are all initialized from the same pretrained encoder $\theta_0 \in \mathbb{R}^d$; fine-tuning on task t gives expert parameters θ_t and task vector $\tau_t = \theta_t - \theta_0$. Writing $r \in \{1, \ldots, d\}$ for a parameter coordinate, the refinement starts from an initial model

$\theta^{(0)} \in \mathbb{R}^d$: any weight-space merge of the experts—task arithmetic

$$\theta^{(0)} = \theta_0 + \sum_{i=1}^{T} \lambda_i \tau_i \tag{1}$$

Let $\mathcal{D}_t^{\text{probe}}$ be a small labeled replay buffer used for refinement, $\mathcal{D}_t^{\text{eval}}$ the evaluation split used only for reporting, and $\mathcal{L}_i(\theta; \mathcal{D}_i^{\text{probe}})$ the replay-buffer loss of encoder parameters θ under task i's prediction head. In the multi-head settings task-specific heads are not merged and the merged encoder is evaluated with each task's own head; the shared-output settings merge everything. The method uses no base-relative saliency term and no pre-merge pruning mask: its only data-dependent operation is a short replay-buffer refinement of $\theta^{(0)}$.

3.2 Expert-Anchored Gradient Descent

The natural data-dependent baseline from this initialization is ordinary replay-buffer gradient descent, $\theta^{(s+1)} = \theta^{(s)} - \eta \sum_{i=1}^{T} \nabla_\theta \mathcal{L}_i(\theta^{(s)}; \mathcal{D}_i^{\text{probe}})$, which treats the merge like any other starting point and lets the replay gradients move every coordinate freely. However, when only a smal replay buffer is available, this can be risky as the gradients may be noisy. Hence, only small learning rates and few steps may be possible. The motivation behind our method is that we can exploit the position of the experts to modify gradient descent to a safer more effective update.

Our method—*expert-anchored gradient descent*—keeps the descent but anchors it to the experts: for each task i and coordinate r we (i) *weight* the negative-gradient step by the current distance $|v_{i,r}|$ to the expert, so coordinates already close to the expert barely move; (ii) *gate* the step, keeping only coordinates where it moves the model *toward* the expert; and (iii) *clip* the result to the current expert distance, so a single update can never overshoot the expert. We call this constraint the *expert-distance cap*: for each coordinate, the interval of radius $|v_{i,r}|$ around the current value, reaching exactly to the expert, which no single update can leave.

Let $\theta^{(s,0)} = \theta^{(s)}$ and let π_s be the task order for sweep s. For within-sweep index $k \in \{1, \ldots, T\}$, set $i = \pi_s(k)$ and compute

$$g_i^{(s,k)} = \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}}), \tag{2}$$

$$v_i^{(s,k)} = \theta_i - \theta^{(s,k-1)}. \tag{3}$$

The update moves each coordinate that passes the gate a capped fraction of its remaining distance to the expert, and leaves every other coordinate unchanged:

$$u_{i,r}^{(s,k)} = \begin{cases} \min\!\big(\eta\,|g_{i,r}^{(s,k)}|,\, 1\big)\, v_{i,r}^{(s,k)} & \text{if } g_{i,r}^{(s,k)}\, v_{i,r}^{(s,k)} < 0, \\ 0 & \text{otherwise;} \end{cases} \tag{4}$$

the model is updated immediately, $\theta^{(s,k)} = \theta^{(s,k-1)} + u_i^{(s,k)}$, with $\theta^{(s+1)} = \theta^{(s,T)}$ after all T tasks in the sweep have been processed. Equation (4) is the weight–gate–clip step of the previous paragraph written in one line: on gated coordinates the sign condition makes $-g_{i,r}^{(s,k)}$ and $v_{i,r}^{(s,k)}$ agree, so the distance-weighted descent step $-\eta\, g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}|$ equals $\eta\, |g_{i,r}^{(s,k)}|\, v_{i,r}^{(s,k)}$, and the expert-distance cap limits the step fraction to one. The gate condition $g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0$ keeps only the coordinates whose replay step also moves the current model toward expert i, and is exactly the sign of the attribution-patching score of Section 3.3 (Equation (5)): the two perspectives agree coordinate by coordinate.

Because the step fraction $\min(\eta\, |g_{i,r}^{(s,k)}|, 1)$ never exceeds one, per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η: the learning rate controls how many coordinates reach the cap, never the maximum step. Optionally, $u_i^{(s,k)}$ is RMS-normalized tensor-wise. Algorithm 1 summarizes the procedure.

Algorithm 1 Sequential expert-anchored gradient descent (APR)

Require: Task experts $\{\theta_i\}_{i=1}^{T}$; task heads; replay buffers $\{\mathcal{D}_i^{\text{probe}}\}_{i=1}^{T}$; initial model $\theta^{(0)}$ (any weight-space merge of the experts, or the pretrained model θ_0 itself); steps S; learning rate η.

for $s = 0, \dots, S-1$ **do**
  $\theta^{(s,0)} \leftarrow \theta^{(s)}$.
  Choose a task order π_s over $\{1, \dots, T\}$.
  for $k = 1, \dots, T$ **do**
    $i \leftarrow \pi_s(k)$.
    $g_i^{(s,k)} \leftarrow \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}})$ using task i's head.
    $v_i^{(s,k)} \leftarrow \theta_i - \theta^{(s,k-1)}$.
    for each mergeable coordinate r **do**
      $u_{i,r}^{(s,k)} \leftarrow \min\big(\eta\,|g_{i,r}^{(s,k)}|,\ 1\big)\, v_{i,r}^{(s,k)} \cdot \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0\}$. {gated, capped step toward θ_i; Eq. (4)}
    end for
    $\theta^{(s,k)} \leftarrow \theta^{(s,k-1)} + u_i^{(s,k)}$.
  end for
  $\theta^{(s+1)} \leftarrow \theta^{(s,T)}$.
end for

return $\theta^{(S)}$.

The informal claim that the refinement "cannot diverge" is made precise by the following guarantee, whose proof (as an instance of a general geometric result, Theorem 1) is given in Appendix A.

Proposition 1 (Non-expansiveness toward the expert box)**.** *Let* $B_{\text{exp}} := \prod_{r=1}^{d} [\min_{j \in \mathcal{T}} \theta_{j,r},\ \max_{j \in \mathcal{T}} \theta_{j,r}]$ *be the axis-aligned box hull of the task experts, and let* $d_p(\cdot, B_{\text{exp}})$ *denote* ℓ^p *distance to it. Then every within-sweep update of Algorithm 1 is non-expansive in distance to the expert box: for every* $1 \le p \le \infty$ *and every sweep* s *and within-sweep index* k*,*

$$d_p\Big(\theta^{(s,k)}, B_{\text{exp}}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\text{exp}}\Big),$$

and hence $d_p(\theta^{(S)}, B_{\text{exp}}) \le d_p(\theta^{(0)}, B_{\text{exp}})$ *after any number of sweeps, at any learning rate.*

Every gated, clipped step moves each coordinate between its current value and the corresponding expert coordinate, and such movement can never increase the distance to the box; the box is moreover exactly the ℓ^1-geodesic convex hull of the experts (Proposition 2), not an arbitrary safe region. The guarantee holds for any initialization $\theta^{(0)}$, including the pretrained model.

3.3 A Second Perspective: Attribution Patching in Parameter Space

In mechanistic interpretability, activation patching measures the effect of an intervention—replacing one internal quantity by its value from a reference run—by executing that intervention; attribution patching replaces the execution with a first-order estimate, the inner product of the local loss gradient with the proposed change, so that a single forward–backward pass scores every candidate patch at once (Syed et al., 2024; Kramar et al., 2024).

The update of Section 3.2 is this construction transported to parameter space. Let ϑ be the current encoder, $v_i(\vartheta) = \theta_i - \vartheta$, and $g_i(\vartheta)$ the replay gradient under task i's head, and take as candidate interventions the d single-coordinate patches that set ϑ_r to the expert value $\theta_{i,r}$. Executing them would cost d evaluations of task i's replay loss; attribution patching instead scores patch r by the r-th term of the directional derivative $\langle g_i(\vartheta), v_i(\vartheta) \rangle$,

$$a_{i,r}(\vartheta) = g_{i,r}(\vartheta)\, v_{i,r}(\vartheta), \tag{5}$$

so one backward pass on the replay buffer prices all d candidates; the score is negative exactly when patching coordinate r toward the expert is predicted to reduce task i's loss.

The score determines every ingredient of Equation (4), not only its gate: on gated coordinates the applied step satisfies

$$|u_{i,r}^{(s,k)}| = \min\big(\eta\,|a_{i,r}(\vartheta)|,\ |v_{i,r}(\vartheta)|\big),$$

so the update is a *partial patch*: it moves exactly the coordinates whose predicted effect is a loss reduction ($a_{i,r} < 0$), each by an amount proportional to its predicted saving $|a_{i,r}|$, and never past the

full patch—beyond the expert value, the score no longer describes the intervention it priced, which is the attribution-patching reading of the expert-distance cap. The score certifies only a local first-order effect for task i; cross-task interference must be measured experimentally. After this reading, we abbreviate the update as *APR* (attribution-patching-guided replay) throughout the experiments.

4 Experimental Setup

Settings. We evaluate on three benchmark families. *Multi-head* GLUE-8 (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE) (Wang et al., 2018) with RoBERTa-base (Liu et al., 2019) merges the shared encoder and evaluates with each task's own trained head—an intentionally *oracle-task-id* setting. A *CLIP/ViT-B/32* vision track uses the standard eight-task task-vector suite (Radford et al., 2021; Ilharco et al., 2022) and its twenty-task extension (Wang et al., 2024), probing a different modality and—since more task vectors then compete for the same encoder weights—a substantially higher-interference regime (Section 5.1). Heads are never fine-tuned, following each benchmark's convention: GLUE-8 keeps each task's trained head, and the CLIP suites keep their frozen text-derived zero-shot heads. Every method receives the same labeled budget of B examples per task ($B \in \{16, 32\}$), drawn from the training data and disjoint from the evaluation split; Appendix C specifies the sampling rule.

Protocol. Every method receives the same B labeled examples per task and never sees a label outside them. Merge baselines choose their hyperparameters (coefficient, density, scale) on all B examples. APR and its ablations split the budget in half: they refine on one half for a fixed horizon of $S = 50$ sweeps, choose the learning rate η on the other half, and are then refit once on all B examples at the chosen η. Selection everywhere uses the tasks' own held-out metrics, averaged over tasks, with ties going to the smaller learning rate (for a merge, the smaller scaling coefficient); grids are extended until the selected cell is interior, the evaluation split is read only for selected cells, and every cell is repeated over three independent buffer draws and reported as mean $\pm$ standard deviation. The selection rule was settled on three diagnostic draws, and every number reported in this paper comes from three fresh draws made afterwards (Appendix C.1); Appendix C gives the grids and the remaining details.

Baselines. We compare against checkpoint-only merges (task arithmetic, TIES, DARE-TIES, Breadcrumbs, DOGE/APGD), merges fitted on unlabeled inputs (AdaMerging, RegMean; given the same B inputs per task that APR sees labeled), and two other ways of spending the same labels: Fisher merging with a Fisher estimated on the replay buffer, and Localize-and-Stitch with masks learned on it (Appendix C). APR's two ablations—ordinary replay gradient descent (GD; no anchor, no gate) and the ungated distance-scaled update (anchor without gate)—share APR's optimizer, schedule, and grid, and are run from the pretrained model, where they isolate what each ingredient contributes; from merge initializations only APR is run.

Metrics. Every suite reports the plain mean over tasks of each task's own metric—top-1 accuracy on the CLIP suites; Matthews correlation (CoLA), the mean of Pearson and Spearman correlation (STS-B), the mean of accuracy and F1 (MRPC, QQP) and accuracy elsewhere on GLUE-8, the benchmark's own convention—with the zero-shot floor and the expert ceiling quoted for scale. Worst-task values accompany every mean, since a merge can raise the average while harming one task badly; on GLUE-8 the worst task is reported on its own metric's scale and is usually CoLA's Matthews correlation. Normalized retention, which rescales each task between the pretrained model and its expert, is defined in Appendix C and used only per task: as an aggregate it lets a task whose expert barely improves on the pretrained model dominate the average.

5 Results

5.1 Performance on the Merged Tasks

Task-level scores at the $8{+}8$ budget are given in Appendix D; the full twenty-task campaign, with its diagnostics and budget grids, is in Appendix E.

Table 1: **Both eight-task suites under the equal-budget protocol** (B labels per task for every method; mean $\pm$ std over three fresh buffer draws). Both suites report the plain mean of the tasks' own metrics (Section 4) and the worst task on the same scale; the expert ceilings are 0.903 (0.749 worst) on CLIP-8 and 0.860 (0.623) on GLUE-8. Every method, merges included, selects its hyperparameters inside the budget by the same held-out-metric rule, so a merge row carries a draw spread wherever its selection varied. Interiority is enforced for η and the task-arithmetic coefficient; TIES, DARE-TIES and Breadcrumbs select on two-point grids per hyperparameter that are not extended. A "+APR" row refines that draw's own selected merge, so it is directly comparable to the merge row above it. GD and ungated are APR's ablations, run from the pretrained model where they isolate the anchor and the gate.

	CLIP-8 (top-1 acc.)		GLUE-8 (task metric)	
Model	Mean	Worst-task	Mean	Worst-task
Pretrained θ_0	0.478	0.315	0.354	-0.047
$B = 32$ *labels per task*				
Task arithmetic	0.685 ± 0.011	0.503 ± 0.026	0.552 ± 0.000	0.130 ± 0.000
Breadcrumbs	0.705 ± 0.000	0.521 ± 0.000	0.473 ± 0.015	0.149 ± 0.012
DARE-TIES	0.708 ± 0.000	0.486 ± 0.000	0.563 ± 0.000	$\mathbf{0.268} \pm 0.000$
TIES	0.732 ± 0.002	0.524 ± 0.018	0.549 ± 0.000	0.234 ± 0.000
Task arithmetic +APR	0.765 ± 0.011	0.557 ± 0.004	0.681 ± 0.023	0.059 ± 0.045
Breadcrumbs +APR	0.766 ± 0.008	0.556 ± 0.006	0.686 ± 0.009	0.096 ± 0.030
DARE-TIES +APR	$\mathbf{0.793} \pm 0.010$	0.556 ± 0.013	$\mathbf{0.699} \pm 0.006$	0.212 ± 0.028
TIES +APR	0.785 ± 0.007	$\mathbf{0.564} \pm 0.005$	0.690 ± 0.010	$\mathbf{0.363} \pm 0.028$
θ_0+GD (ablation)	0.546 ± 0.032	0.368 ± 0.041	0.521 ± 0.017	0.111 ± 0.068
θ_0+ungated (ablation)	0.579 ± 0.009	0.443 ± 0.012	0.480 ± 0.060	0.115 ± 0.074
θ_0+APR	0.751 ± 0.004	0.524 ± 0.003	0.612 ± 0.015	0.133 ± 0.047
$B = 16$ *labels per task* (compositions not yet run at this budget)				
Task arithmetic	0.685 ± 0.011	0.503 ± 0.026	0.552 ± 0.000	0.130 ± 0.000
Breadcrumbs	0.705 ± 0.000	$\mathbf{0.521} \pm 0.000$	0.470 ± 0.020	0.132 ± 0.017
DARE-TIES	0.708 ± 0.000	0.486 ± 0.000	0.554 ± 0.016	$\mathbf{0.256} \pm 0.021$
TIES	$\mathbf{0.734} \pm 0.007$	0.512 ± 0.014	0.549 ± 0.000	0.234 ± 0.000
θ_0+GD (ablation)	0.525 ± 0.036	0.351 ± 0.032	0.395 ± 0.048	0.011 ± 0.064
θ_0+ungated (ablation)	0.548 ± 0.006	0.402 ± 0.010	0.456 ± 0.008	0.066 ± 0.055
θ_0+APR	0.681 ± 0.050	0.502 ± 0.020	$\mathbf{0.565} \pm 0.032$	0.062 ± 0.051

Table 2: **CLIP-20 under the equal-budget protocol**, same conventions as Table 1: top-1 accuracy averaged over the twenty tasks (zero-shot floor 0.550, worst task 0.100; expert ceiling 0.896, worst 0.711). The task-arithmetic grid starts at $\lambda = 0.05$ here and every selected λ is interior; the ungated ablation's rate reached the lower edge of its twice-extended grid on two draws at $B = 32$.

	$B = 32$		$B = 16$	
Model	Mean	Worst-task	Mean	Worst-task
Pretrained θ_0	0.550	0.100	0.550	0.100
Task arithmetic	0.604 ± 0.000	0.115 ± 0.000	0.602 ± 0.004	0.117 ± 0.004
Breadcrumbs	0.598 ± 0.009	0.133 ± 0.024	0.603 ± 0.000	0.119 ± 0.000
DARE-TIES	0.520 ± 0.000	0.202 ± 0.000	0.520 ± 0.000	$\mathbf{0.202} \pm 0.000$
TIES	0.619 ± 0.000	0.194 ± 0.000	0.612 ± 0.013	0.201 ± 0.013
θ_0+GD (ablation)	0.573 ± 0.021	0.103 ± 0.005	0.563 ± 0.016	0.107 ± 0.013
θ_0+ungated (ablation)	0.602 ± 0.005	0.156 ± 0.011	0.578 ± 0.003	0.151 ± 0.007
θ_0+APR	$\mathbf{0.663} \pm 0.009$	$\mathbf{0.267} \pm 0.029$	$\mathbf{0.613} \pm 0.056$	0.188 ± 0.083

The underlying task-level evaluation scores for the two eight-task encoder benchmarks at the $8{+}8$ budget are reported in Appendix Tables 7 and 8.

Reading the grid. Table 3 reports the three encoder benchmarks at a labeled budget of 16 refinement and 16 selection examples per task; Table 4 repeats them at half that budget. On CLIP-20 (zero-shot

Table 3: **Results at the** $16{+}16$ **budget.** Mean (worst-task) normalized retention on GLUE-8 and CLIP-8, absolute top-1 accuracy on CLIP-20; every hyperparameter selected on a labeled buffer disjoint from the refinement buffer, evaluation split read only at selected cells. Bottom block: other uses of the same labels. †: selected S at the top of its grid. Dashes: not run.

Initialization	Treatment	GLUE-8 normalized retention	CLIP-8 normalized retention	CLIP-20 top-1 acc.
Pretrained	alone	0	0	0.550 (.10)
	+GD	0.209 (−.18)	0.046† (−.36)	0.583† (.12)
	+ungated	−0.018 (−.49)	−0.036 (−.59)	0.586 (.11)
	+APR	0.315 (−.12)	0.432 (−.16)	0.647 (.22)
Task arithmetic	alone	0.320 (−.18)	0.270 (−.53)	0.495 (.15)
	+APR	**0.608** (+.10)	0.524† (+.05)	0.667 (.25)
Breadcrumbs	alone	0.030 (−2.14)	0.431 (+.08)	0.603 (.12)
	+APR	0.567 (+.04)	0.485† (+.00)	0.661 (.20)
DARE-TIES	alone	0.301 (−.34)	0.364 (−.32)	0.520 (.20)
	+APR	0.586 (+.11)	0.527† (−.39)	0.664 (.30)
TIES	alone	0.208 (−.22)	0.525 (+.21)	0.619 (.19)
	+APR	0.543 (−.03)	0.608† (+.11)	**0.676** (.27)
DOGE/APGD	alone	0.016 (+.00)	0.138 (−1.93)	0.516 (.16)
AdaMerging	alone	0.029 (−3.02)	0.533 (+.09)	0.597 (.10)
	+APR	0.564 (−.02)	**0.638**† (+.32)	0.665 (.20)
Labeled alt.	Fisher (replay)	−0.053 (−2.84)	0.069 (−.49)	0.585 (.11)
	L&S (learned)	0.230 (−.47)	0.295 (−.15)	0.600 (.14)

Table 4: **Results at the** $8{+}8$ **budget.** Same conventions as Table 3.

Initialization	Treatment	GLUE-8 normalized retention	CLIP-8 normalized retention	CLIP-20 top-1 acc.
Pretrained	alone	0	0	0.550 (.10)
	+GD	−0.361 (−3.52)	−0.148 (−1.04)	0.574† (.10)
	+ungated	0.089† (−.31)	−0.006 (−.41)	0.568 (.08)
	+APR	0.223† (−.33)	0.295† (−.45)	0.645 (.22)
Task arithmetic	alone	0.320 (−.18)	0.270 (−.53)	0.495 (.15)
	+APR	0.498 (−.18)	0.462† (−.12)	0.648 (.22)
Breadcrumbs	alone	0.030 (−2.14)	0.431 (+.08)	0.602 (.12)
	+APR	0.528 (−.06)	0.339† (−.46)	0.652 (.22)
DARE-TIES	alone	0.301 (−.34)	0.364 (−.32)	0.520 (.20)
	+APR	**0.576** (+.19)	0.498† (−.31)	0.649† (.27)
TIES	alone	0.208 (−.22)	0.525 (+.21)	0.619 (.19)
	+APR	0.489† (+.01)	0.566† (−.08)	0.665 (.28)
DOGE/APGD	alone	0.016 (+.00)	0.138 (−1.93)	0.516 (.16)
AdaMerging	alone	0.028 (−3.01)	0.463 (−.04)	0.592 (.10)
	+APR	0.543 (+.02)	0.554† (+.15)	0.640 (.19)
RegMean	alone	0.477 (+.06)	0.627 (+.32)	0.697 (.21)
	+APR	0.517 (+.10)	**0.632**† (+.31)	0.709 (.24)

floor 0.550, expert ceiling 0.896) interference dominates: the best checkpoint-only merge, TIES, recovers only a fifth of the floor-to-ceiling gap, against roughly half of the expert–base gap at eight tasks, which Appendix E traces to the geometry of the twenty-expert compromise. Its task-arithmetic row carries an artifact: the coefficient grid used for selection, inherited unchanged from the eight-task runs ($\lambda \in \{0.2, \ldots, 0.5\}$), did not reach the twenty-task optimum ($\lambda \approx 0.05$–0.08; Appendix E), so the selected $\lambda = 0.2$ is a boundary pick and that merge lands below the zero-shot floor (0.495).

Checkpoint-only and data-fitted baselines. DOGE/APGD is the nearest neighbour of our update in form—it, too, improves a merge by gradient descent—but its objective is data-free, so the replay buffer plays no part in constructing it and the labeled budget enters only through the selection of its global scale η. The $8{+}8$ and $16{+}16$ selection buffers choose the same η on every benchmark (0.1, 0.09, and 0.03 on GLUE-8, CLIP-8, and CLIP-20, each interior to its grid), which is why its row is

identical in Tables 3 and 4. At those scales it is not competitive: 0.016 mean retention on GLUE-8, 0.138 on CLIP-8 with a worst task of -1.93, and 0.516 accuracy on CLIP-20—below every other merge in its column except the boundary-selected task-arithmetic row. Whether its published default scale would fare better is unknown by construction: only the selected cell is ever evaluated, and substituting another scale after seeing test scores would be exactly the evaluation-split tuning the protocol forbids. The row is therefore a statement about DOGE/APGD under split selection at this budget, not about its best achievable performance.

RegMean supplies a substantially stronger data-fitted starting point at the $8{+}8$ budget: its mean retention is 0.477 on GLUE-8 and 0.627 on CLIP-8, and its mean accuracy is 0.697 on CLIP-20. Refinement improves it on all three suites, but by less than it improves checkpoint-space merges: APR reaches 0.517 and 0.632 retention on GLUE-8 and CLIP-8, while on CLIP-20 APR and the ungated anchored update tie at 0.709 mean accuracy. The ordinary-GD cell is effectively unchanged on the retention benchmarks and trails both anchored arms on CLIP-20. The selected RegMean refinement rates (0.5, 1, and 0.125 for APR) are smaller than those used from checkpoint-space initializations, consistent with the far-initialization behavior examined in Appendix E.

Cold start under the equal-budget protocol. Table 1 reports both eight-task suites under the protocol of Section 4: every method given the same B labels per task, hyperparameters selected inside that budget by one rule, and every number a mean over three fresh buffer draws. On CLIP-8 at $B = 32$, APR from the pretrained model reaches 0.751 ± 0.004 mean accuracy, above every checkpoint-only merge including TIES (0.732), while using 32 labels per task in place of the eight expert checkpoints a merge requires; it selects the same interior cell ($\eta = 32$) on every draw. Its two ablations sit at 0.546 and 0.579—barely above the zero-shot floor of 0.478 once one accounts for the merges' head start—so the anchored, capped step is worth 0.17–0.21 accuracy over unconstrained descent from the same initialization, and the anchor without the gate does not account for it.

On GLUE-8 the ordering is the same against the ablations and, at both budgets, against the merges as well. APR reaches 0.612 ± 0.015 at $B = 32$ against 0.563 for the best checkpoint-only merge (DARE-TIES), 0.521 for ordinary descent and 0.480 for the ungated update, and 0.565 ± 0.032 at $B = 16$ against 0.554, 0.395 and 0.456. The worst-task column tells the other half of the story: APR's mean is bought partly at the expense of its weakest task (0.133 against DARE-TIES's 0.268 at $B = 32$), so on this suite the refinement trades the worst task for the average where the merges do not, and every selected cell sits in the high-displacement part of the grid ($\eta \in \{8, 16, 32\}$, 2.6–6.1 from θ_0). The selection rule is what makes GLUE-8 reportable at all. The same experiment selected by held-out replay loss chose a near-no-op on roughly half of its draws and produced a ± 0.11 spread, because RoBERTa's pretrained encoder predicts near-uniformly under every GLUE-8 head—the initial held-out loss is $\approx \ln C$—and an objective that is already near its normalized value of 1 prefers models that do not move over models that move far enough to score; Appendix C.1 documents the failure, the pinned cells that diagnose it, and the rule that replaced it.

On CLIP-8 the draw spread is the more interesting number. At $B = 32$ APR varies by ± 0.004 across buffer draws, the same order as the merges, and selects the same cell every time. At $B = 16$ it does not: two draws select $\eta \in \{4, 8\}$ and score 0.63–0.67, the third selects $\eta = 32$ and scores 0.73, and the mean (0.681 ± 0.050) lands below TIES (0.734). This is the one cell of Table 1 in which a checkpoint-only merge beats the refinement, and Appendix C.1 traces it to the select-then-refit step rather than to the update: cells are constructed on $B/2 = 8$ examples per task, where $\eta = 32$ genuinely collapses on some draws, and refit on 16, where it is safe—pinning $\eta = 32$ on the two draws that rejected it scores 0.725 and 0.732. Selection is right about the models it can see and conservative about the model it will build. An earlier version of the $B = 32$ experiment showed a different instability—a ± 0.081 spread with the fine-grained Cars task collapsing on two draws of three—which we traced to the replay sampler: with more classes than the budget holds it had been keeping the lowest class ids deterministically, so Cars was refined and selected on 32 of its 196 classes, identically for every seed. Drawing the covered classes at random per seed (Appendix C) removes both the collapse and the spread. We report this because the failure is instructive: at these budgets a fine-grained task is representable only in part, and a selection buffer that inherits the same blind spot cannot detect damage to the classes it never sees.

Refining a merge under the same protocol. The same rule, unchanged, also applies to refinement from a merge (Table 1, "+APR" rows, $B = 32$). Every composition improves its own initialization on

every draw: on CLIP-8 by 0.046–0.104 and on GLUE-8 by 0.103–0.220, lifting the best cell to 0.793 (DARE-TIES+APR) and 0.699 (DARE-TIES+APR) respectively, above APR from the pretrained model on both suites. Refinement also compresses the spread between merges: CLIP-8's four merges span 0.685–0.732 before refinement and 0.765–0.793 after, and GLUE-8's span 0.473–0.563 before and 0.681–0.699 after, so which merge one starts from matters less once the labels are spent. The selected rates are an order of magnitude smaller than from the pretrained model ($\eta \in [0.25, 4]$ on CLIP-8 against $\eta = 32$; one TIES draw selects at the twice-extended lower edge of the grid), which is what a starting point that already carries multi-task signal should require. Worst-task behaviour splits by initialization on GLUE-8: TIES+APR raises the weakest task from 0.234 to 0.363, while task arithmetic+APR lowers it from 0.130 to 0.059—the mean gain there is bought partly from the weakest task, as it is for APR from the pretrained model.

Twenty tasks. On CLIP-20 (Table 2) interference dominates: with twenty task vectors competing for one encoder, the best merge, TIES, recovers a fifth of the floor-to-ceiling gap (0.619) against half on CLIP-8, and the selected task-arithmetic coefficient falls to $\lambda \in \{0.05, 0.08\}$ (Appendix E). APR from the pretrained model reaches 0.663 ± 0.009 at $B = 32$, above every merge and both ablations on the mean and on the worst task (0.267), with 32 labels per task in place of twenty expert checkpoints. At $B = 16$ it ties TIES (0.613 ± 0.056 against 0.612), and the spread is again selection: the three draws select $\eta \in \{4, 8, 16\}$ and score 0.55–0.65, and the draw that chose $\eta = 16$ scored best on its eight construction examples and worst after the refit on sixteen—at this budget, eight examples per task are an unreliable guide to the refit in either direction. The selected rates ($\eta \leq 16$) are smaller than CLIP-8's $\eta = 32$ on every draw.

Cold start: refining from the pretrained model. Refining *from the pretrained model*—the setting with no merge to inherit, and the one where the refinement must supply all of the multi-task ability itself—is an unambiguous win for the gated, expert-anchored update: APR is the best cold-start treatment on all three benchmarks, ahead of the better of the two unconstrained baselines by 0.06 (CLIP-20), 0.11 (GLUE-8), and 0.39 (CLIP-8). Neither unconstrained baseline is viable at this budget: ordinary descent recovers between a twentieth and a fifth of the expert–base gap on the two retention benchmarks (0.046 on CLIP-8, 0.209 on GLUE-8), and the ungated update fails to improve on the pretrained model at all (-0.018 on GLUE-8, -0.036 on CLIP-8), so distance scaling without the gate is not merely weaker than APR but useless from this initialization. Only the gated, distance-scaled update turns a small replay buffer into multi-task ability with no merge to build on.

Composition: refining from a merge. From merge initializations the refinement is a complement to merging rather than a replacement. At the $16{+}16$ budget it improves every initialization on all three encoder benchmarks. Refinement also makes the outcome far less sensitive to which merge it was handed: on CLIP-20 the unrefined merges span 0.495–0.619 while the +APR rows span 0.661–0.676, and on GLUE-8 a spread of 0.029–0.320 closes to 0.543–0.608. The ablation arms are reported only from the pretrained model, where they isolate the contribution of the anchor and the gate; from a merge the comparison of interest is against the merge itself, which every composition row makes directly.

Among the alternative uses of the same labels (bottom block of Table 3), the stronger is Localize-and-Stitch with masks learned on the buffer, at 0.230/0.295 retention on GLUE-8/CLIP-8 and 0.600 accuracy on CLIP-20; Fisher merging with a replay-estimated Fisher is weaker still. Both fall well below the composition tier of the same table—every merge-initialized APR row exceeds 0.543 on GLUE-8, 0.485 on CLIP-8, and 0.661 on CLIP-20—so at this sample size the labels are better spent refining the encoder than learning masks or Fisher weights.

5.2 Retention of Pretrained Capabilities

Strong performance on the merged tasks is useful only if it does not erase capabilities inherited from the pretrained model. We test this first on the pretraining objective itself and then on tasks that are never merged or used for refinement.

Drift on the pretraining objective. Figure 1 shows the refinement reaching merge-level scores at a small fraction of the merge's displacement, but parameter distance is a geometric quantity, not a

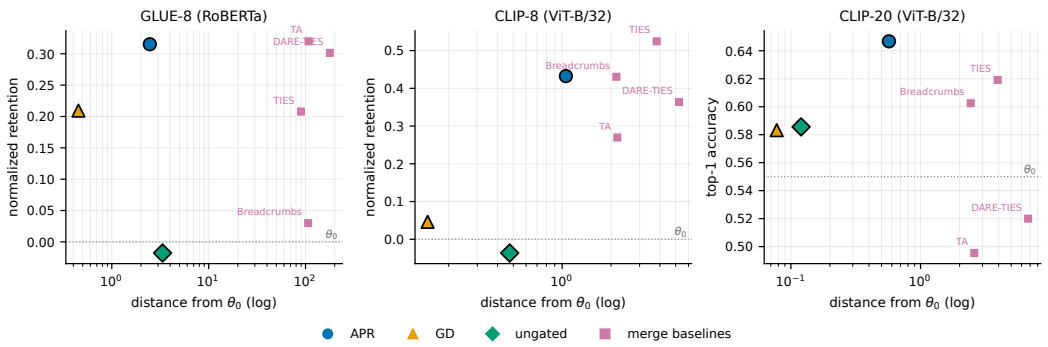

Figure 1: Task performance versus distance from the pretrained model θ_0. Circles, triangles, and diamonds show split-selected APR, GD, and ungated refinement from θ_0; squares show the selected merge baselines. The horizontal dotted line marks pretrained performance, and distance is shown on a log scale.

Table 5: Pretraining-objective drift on GLUE-8 at the B=32 budget of Table 1. MLM loss is measured on WikiText-2 validation with RoBERTa's pretrained MLM head; Δ is relative to θ_0. Every probed state is the model Table 1 scores (the saved winner of each of the three fresh draws, and the merges at their selected hyperparameters), so the GLUE-8 column repeats that table's mean of per-task metrics; mean $\pm$ std over draws. * marks the score–drift Pareto frontier.

State	Dist. θ_0	GLUE-8 (task metric)	MLM loss (Δ)
Pretrained $\theta_0{}^*$	0.00	0.354	2.51 (0.00)
Task arithmetic (λ=0.3)	107.7	0.552 ± 0.000	5.02 (+2.51)
TIES*	90.0	0.549 ± 0.000	**2.95** (+0.44)
DARE-TIES	178.3	0.563 ± 0.000	4.10 (+1.59)
Breadcrumbs	101.0	0.473 ± 0.015	3.44 (+0.93)
APR from $\theta_0{}^*$	3.54 ± 1.28	$\mathbf{0.612} \pm 0.015$	3.44 ± 0.31 (+0.93)
GD ablation	0.40 ± 0.01	0.521 ± 0.017	3.56 ± 0.07 (+1.05)
ungated ablation	1.95 ± 0.98	0.480 ± 0.060	7.89 ± 6.01 (+5.38)

capability measure. We therefore measure drift on the pretraining objective itself: masked-language-model loss on WikiText-2 validation text. Each refined encoder is evaluated under RoBERTa's original pretrained MLM head; the head is never modified by any experiment, and the WikiText-2 text is never used for refinement or selection, so any increase in this loss over the pretrained model is attributable to encoder drift alone. The probed states are exactly the from-pretrained cells of Table 1 at $B = 32$—the saved winner of each of the three fresh draws, nothing re-derived—with the pretrained model and the merges at their selected hyperparameters as references (Table 5).

With the selected merges probed alongside (Table 5), the picture parallels the held-out-task study below. APR is the only point that combines the best task score with sub-merge drift: at 0.612 it exceeds every merge (DARE-TIES 0.563, task arithmetic 0.552) while adding +0.93 to the pretraining loss against their +1.59 and +2.51—a better multitask model for 37% of task arithmetic's damage, at roughly 3% of its parameter displacement—and it dominates Breadcrumbs outright (the same drift at 0.14 more score). The exception is TIES, which drifts least of every merge (+0.44) at 0.063 less score, so the frontier consists of θ_0, TIES and APR alone. The ablations locate the drift protection: ordinary descent barely moves (0.40 from θ_0) and still drifts more than APR, while the ungated update, which moves less far than APR on average, adds +12.3 to the loss on one draw of three—distance scaling without the gate can leave the encoder while barely displacing it. Refinement from the pretrained model is thus not the minimum-drift point, but it is the only one at the high-score end that costs the base model less than half of what a merge at that level does.

We also test whether the methods preserve zero-shot capabilities on tasks excluded from merging and refinement. Models are built on the standard CLIP-8 suite under the protocol of Table 1 at $B = 32$

Table 6: Held-out zero-shot retention after building models on CLIP-8 at B=32. Every state is the model Table 1 scores—the saved winner of each fresh draw, and the merges at their selected hyperparameters—evaluated with no further training on the seven CLIP-20 tasks on which θ_0 performs meaningfully above chance; parentheses report the change from θ_0; mean $\pm$ std over three draws. * marks the train–held-out Pareto frontier.

Model (built from the 8 train tasks)	Dist. θ_0	Train-8	Held-out (drop)
Base model θ_0*	0.00	0.478	0.780 (0.000)
GD from base*	0.04 ± 0.03	0.546 ± 0.032	$\mathbf{0.773} \pm 0.008$ (-0.006)
Ungated from base*	0.30 ± 0.06	0.579 ± 0.009	0.746 ± 0.009 (-0.034)
APR from base*	1.05 ± 0.02	$\mathbf{0.751} \pm 0.004$	0.721 ± 0.009 (-0.059)
Task arithmetic	1.70 ± 0.42	0.685 ± 0.011	0.695 ± 0.058 (-0.085)
Breadcrumbs	2.16 ± 0.00	0.705 ± 0.000	0.716 ± 0.000 (-0.064)
TIES*	3.08 ± 0.04	0.732 ± 0.002	0.724 ± 0.030 (-0.056)
DARE-TIES	5.25 ± 0.00	0.708 ± 0.000	0.613 ± 0.000 (-0.166)

and evaluated, with no further training or selection, on seven additional CLIP-20 tasks on which the pretrained model performs meaningfully above chance; every probed state is the saved winner of a fresh draw or a merge at its selected hyperparameters, so nothing is re-derived. The full twelve-task split and the five near-chance tasks are given in Appendix E.

APR scores highest on the train tasks (0.751) and retains 0.721 of held-out accuracy (a drop of 0.059), which matches the best merge, TIES (0.724, inside a single draw's spread), at a third of its displacement and 0.019 more train accuracy; every other merge is dominated—Breadcrumbs (0.716), task arithmetic (0.695) and DARE-TIES (0.613) retain less at a lower train score. The two ablations retain more (0.773 and 0.746), but they do so by barely moving—ordinary descent stays within 0.04 of θ_0 and scores 0.21 less on the train tasks—so the frontier consists of θ_0, the two ablations, APR and TIES, and retention is cheap for a model that has not learned the tasks. What the ablations do show is that APR's retention is not a consequence of its labels alone: it gains 0.17–0.21 train accuracy over both while giving up 0.03–0.05 of held-out accuracy, whereas every merge but TIES gives up 0.06–0.17 for a lower train score.

Summary. Five claims survive the campaign at $n \leq 16$: (i) the refinement is broadly compositional, improving most initializations, while the small-buffer Breadcrumbs rows delimit any universal improvement claim; (ii) the anchored, distance-scaled, clipped step is the load-bearing ingredient, the gate contributing worst-task repair, small-budget stability, and held-out retention rather than mean accuracy; (iii) in the dedicated safety audit, anchored runs avoid the overfitting signature that appears in a quarter of unconstrained descent's small-buffer configurations, although selected composition rows can still decline; (iv) eight labels per task suffice to beat the tuned task-arithmetic merge starting from the pretrained model alone; and (v) on held-out tasks never merged or refined, the anchored update retains as much as the best checkpoint-only merge while scoring higher on the merged tasks, and more than every other merge; its unconstrained controls retain more only by staying near the pretrained model (Table 6).

6 Discussion

We introduced APR, an expert-anchored refinement method designed to repair task interference in model merges. APR uses replay gradients to identify locally beneficial movements toward each task expert, scales those movements by the expert distance, and clips them before they can overshoot. Under the equal-budget protocol APR improves every checkpoint-only merge it is given, on every buffer draw, on both eight-task suites; when initialized directly from the pretrained model, it is the strongest of the tested replay refinements on all three benchmarks and on every buffer draw, and under the equal-budget protocol it outperforms every checkpoint-only merge on GLUE-8 at both budgets and on CLIP-8 at $B = 32$, trailing TIES on CLIP-8 at $B = 16$ because a learning rate selected on eight examples per task is conservative about the step the refit on sixteen can afford. The ablations identify the anchored, distance-scaled, clipped update as the main source of the gains, with the gate contributing primarily to worst-task performance and retention. On tasks excluded from merging and refinement, APR retains as much zero-shot ability as the best checkpoint-only merge while scoring higher on the merged tasks, and more than every other merge; its unconstrained controls retain more

only by staying near the pretrained model. APR is therefore best viewed as a practical correction stage for model merging when a modest labeled buffer is available, with the option to operate directly from the pretrained model when no suitable merge exists. Establishing whether these conclusions extend beyond the encoder architectures and small-data regimes studied here remains future work.

AI USE STATEMENT

In this work, we used generative AI tools to assist with writing and refactoring experiment code and with editing and polishing the text. We did not use generative AI tools for research ideation, experimental design, or data analysis. We have reviewed all AI-assisted work and take responsibility for the final content of this work, including any text, claims, or artifacts produced with the aid of generative AI.

REFERENCES

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. On tiny episodic memories in continual learning. *arXiv preprint arXiv:1902.10486*, 2019.

Mohammad Davari and Eugene Belilovsky. Model Breadcrumbs: Scaling multi-task model merging with sparse masks. *arXiv preprint arXiv:2312.06795*, 2023.

Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. DELLA-Merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024.

Shwai He, Runqi Wang, Jindong Wang, Jindong Gu, and Xing Xie. Localize-and-Stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024.

Yifei He, Siqi Zeng, Yuzheng Hu, Rui Yang, Tong Zhang, and Han Zhao. MergeBench: A benchmark for merging domain-specialized LLMs. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *International Conference on Learning Representations*, 2023.

Fanshuang Kong, Richong Zhang, and Ziqiao Wang. Activated parameter locating via causal intervention for model merging. *arXiv preprint arXiv:2408.09485*, 2024.

Janos Kramar, Tom Lieberum, Rohin Shah, and Neel Nanda. AtP*: An efficient and scalable method for localizing language model behavior to components. *arXiv preprint arXiv:2403.00745*, 2024.

Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations*, 2022.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Zhenyi Lu, Chengxu Yin, Wujie Wang, and Xuebo Liu. Twin-Merging: Dynamic integration of modular expertise in model merging. *arXiv preprint arXiv:2406.15479*, 2024.

Michael S. Matena and Colin A. Raffel. Merging models with Fisher-weighted averaging. In *Advances in Neural Information Processing Systems*, 2022.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

Anthony Robins. Catastrophic forgetting, rehearsal and pseudorehearsal. *Connection Science*, 7(2):123–146, 1995.

Wenju Sun, Qingyong Li, Wen Wang, Yangli-ao Geng, and Boyang Li. Task Arithmetic in Trust Region: A training-free model merging approach to navigate knowledge conflicts. *arXiv preprint arXiv:2501.15065*, 2025.

Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2024.

Anke Tang, Li Shen, Yong Luo, Liang Ding, Han Hu, Bo Du, and Dacheng Tao. FusionBench: A comprehensive benchmark of deep model fusion. *arXiv preprint arXiv:2406.03280*, 2024.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, 2019.

Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jiménez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. In *International Conference on Machine Learning*, 2024.

Yongxian Wei, Anke Tang, Li Shen, Zixuan Hu, Chun Yuan, and Xiaochun Cao. Modeling multi-task model merging as adaptive projective gradient descent. In *International Conference on Machine Learning*, 2025.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, 2022.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Xingrun Xing. LM-Cocktail: Resilient tuning of language models via model merging. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.

Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2024.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are Super Mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*, 2023.

Zihan Zeng, Jiahui Chen, Lei Li, and Min Zhang. Efficient model editing with task vector bases: A theoretical framework and scalable approach. *arXiv preprint arXiv:2502.01015*, 2025.

A BOX-DISTANCE MONOTONICITY AND THE EXPERT BOX

This appendix proves the general geometric result behind Corollary 1. The statement is self-contained: $x_1, \dots, x_n$ are arbitrary points in $\mathbb{R}^D$ (in the application, the task experts $\theta_1, \dots, \theta_T$), and the superscript d indexes coordinates.

Theorem 1 (Box-distance monotonicity under coordinatewise movement toward a hull point)**.** *Let $x_1, \dots, x_n \in \mathbb{R}^D$. Define the axis-aligned box hull*

$$B := \prod_{d=1}^{D} [m_d, M_d], \qquad m_d := \min_{1 \le j \le n} x_j^d, \qquad M_d := \max_{1 \le j \le n} x_j^d.$$

Let $y, z \in \mathbb{R}^D$. Suppose that there exists some $i \in \{1, \dots, n\}$ such that, for every coordinate $d \in \{1, \dots, D\}$, the point z^d lies between x_i^d and y^d. Equivalently,

$$|z^d - x_i^d| \le |y^d - x_i^d|$$

and $z^d - x_i^d$ has the same sign as $y^d - x_i^d$, allowing equality.

Then, for every $1 \le p \le \infty$,

$$d_p(z, B) \le d_p(y, B),$$

where $d_p(u, B)$ denotes the distance from u to B with respect to the ℓ^p-norm.

Proof. For each coordinate d, set

$$I_d := [m_d, M_d].$$

Since $x_i \in B$, we have

$$x_i^d \in I_d$$

for every d.

Define the one-dimensional distance function

$$\delta_d(t) := \operatorname{dist}(t, I_d) = \inf_{s \in I_d} |t - s|.$$

We first prove the following elementary one-dimensional claim.

Claim. Let $I = [m, M] \subset \mathbb{R}$, let $c \in I$, and suppose that z lies between c and y. Then

$$\operatorname{dist}(z, I) \le \operatorname{dist}(y, I).$$

Indeed, if $y \in I$, then since $c \in I$ and I is an interval, every point between c and y also lies in I. Hence $z \in I$, so

$$\operatorname{dist}(z, I) = 0 = \operatorname{dist}(y, I).$$

If $y > M$, then

$$\operatorname{dist}(y, I) = y - M.$$

Since z lies between $c \in I$ and $y > M$, we have $z \le y$. If $z \le M$, then $\operatorname{dist}(z, I) = 0$. If $z > M$, then

$$\operatorname{dist}(z, I) = z - M \le y - M = \operatorname{dist}(y, I).$$

The case $y < m$ is symmetric. This proves the claim.

Now apply the claim coordinatewise with

$$I = I_d, \qquad c = x_i^d.$$

Because z^d lies between x_i^d and y^d, we obtain

$$\delta_d(z^d) \le \delta_d(y^d)$$

for every $d \in \{1, \dots, D\}$.

Next, distance to an axis-aligned box separates coordinatewise. Namely, for $1 \le p < \infty$,

$$d_p(u, B) = \left(\sum_{d=1}^{D} \delta_d(u^d)^p \right)^{1/p},$$

and for $p = \infty$,

$$d_\infty(u, B) = \max_{1 \le d \le D} \delta_d(u^d).$$

This follows because the closest point of B to u is obtained by clipping each coordinate u^d to the interval $I_d = [m_d, M_d]$.

Since

$$\delta_d(z^d) \le \delta_d(y^d)$$

for every d, monotonicity of the ℓ^p-norm on the nonnegative orthant gives

$$d_p(z, B) \le d_p(y, B).$$

This proves the theorem. □

The box hull B may look like a loose relaxation of the ordinary convex hull of the experts, but it is canonical in its own right: it is the smallest set containing the experts that is convex with respect to ℓ^1-geodesics.

Proposition 2. *The set B is the ℓ^1-geodesic convex hull of $\{x_1, \dots, x_n\}$.*

Proof. For $a, b \in \mathbb{R}^D$, define the ℓ^1-metric interval between a and b by

$$I_1(a, b) := \{u \in \mathbb{R}^D : \|a - u\|_1 + \|u - b\|_1 = \|a - b\|_1\}.$$

We claim that

$$I_1(a, b) = \prod_{d=1}^{D} [\min(a^d, b^d), \max(a^d, b^d)].$$

Indeed, by the triangle inequality in one dimension,

$$|a^d - u^d| + |u^d - b^d| \ge |a^d - b^d|$$

for each coordinate d. Summing over d, equality in the ℓ^1-triangle inequality holds if and only if equality holds in every coordinate. In one dimension, equality holds precisely when u^d lies between a^d and b^d. Hence the displayed formula for $I_1(a, b)$ follows.

Now B is clearly ℓ^1-geodesically convex: if $a, b \in B$, then every point coordinatewise between a and b also lies in B.

It remains to show minimality. Let $C \subseteq \mathbb{R}^D$ be any ℓ^1-geodesically convex set containing $x_1, \dots, x_n$. Since C is closed under ℓ^1-metric intervals, it is closed under taking coordinatewise boxes between pairs of its points. In particular, if $a, b \in C$, then both the coordinatewise minimum

$$a \wedge b := (\min(a^1, b^1), \dots, \min(a^D, b^D))$$

and the coordinatewise maximum

$$a \vee b := (\max(a^1, b^1), \dots, \max(a^D, b^D))$$

belong to C.

By induction, C therefore contains the coordinatewise minimum

$$m := (m_1, \dots, m_D)$$

and the coordinatewise maximum

$$M := (M_1, \dots, M_D)$$

of the points $x_1, \dots, x_n$. Since C is ℓ^1-geodesically convex, it contains the entire ℓ^1-metric interval between m and M. But

$$I_1(m, M) = \prod_{d=1}^{D} [m_d, M_d] = B.$$

Therefore every ℓ^1-geodesically convex set containing $x_1, \dots, x_n$ also contains B. Hence B is the smallest such set, i.e.

$$B = \mathrm{gconv}_{\ell^1}\{x_1, \dots, x_n\}.$$

□

We can now state and prove the guarantee for Algorithm 1 used in Section 3.2.

Corollary 1 (Box-distance monotonicity of the sequential refinement)**.** *Let*

$$B_{\exp} := \prod_{r=1}^{d} [\underline{\theta}_r, \overline{\theta}_r], \qquad \underline{\theta}_r := \min_{j \in \mathcal{T}} \theta_{j,r}, \qquad \overline{\theta}_r := \max_{j \in \mathcal{T}} \theta_{j,r},$$

be the axis-aligned box hull of the task experts. Then every within-sweep update of Algorithm 1 is non-expansive in distance to the expert box: for every $1 \le p \le \infty$*,*

$$d_p\Big(\theta^{(s,k)}, B_{\exp}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\exp}\Big).$$

Consequently, after any number S *of sweeps,*

$$d_p\Big(\theta^{(S)}, B_{\exp}\Big) \le d_p\Big(\theta^{(0)}, B_{\exp}\Big) \qquad \textit{for every } 1 \le p \le \infty.$$

Proof. Fix a sweep s, a within-sweep index k, and let $i = \pi_s(k)$. We first verify the hypothesis of Theorem 1 with

$$y = \theta^{(s,k-1)}, \qquad z = \theta^{(s,k)}, \qquad x_i = \theta_i.$$

It is enough to check that, for every coordinate r, the new coordinate $\theta_r^{(s,k)}$ lies between the previous coordinate $\theta_r^{(s,k-1)}$ and the expert coordinate $\theta_{i,r}$.

By Equation (4), every coordinate update has the form

$$u_{i,r}^{(s,k)} = \alpha_{i,r}^{(s,k)} v_{i,r}^{(s,k)}, \qquad \alpha_{i,r}^{(s,k)} = \begin{cases} \min\big(\eta\, |g_{i,r}^{(s,k)}|,\, 1\big) & \text{if } g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0, \\ 0 & \text{otherwise,} \end{cases}$$

with $\alpha_{i,r}^{(s,k)} \in [0, 1]$ in both cases since $\eta \ge 0$. The same representation is assumed directly in the optional post-processing case. Hence

$$\theta_r^{(s,k)} = \theta_r^{(s,k-1)} + \alpha_{i,r}^{(s,k)} \big(\theta_{i,r} - \theta_r^{(s,k-1)}\big),$$

so $\theta_r^{(s,k)}$ lies between $\theta_r^{(s,k-1)}$ and $\theta_{i,r}$.

The box $B_{\exp}$ is exactly the axis-aligned hull of the expert points $\theta_1, \dots, \theta_T$. Therefore Theorem 1 gives

$$d_p\Big(\theta^{(s,k)}, B_{\exp}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\exp}\Big)$$

for every $1 \le p \le \infty$. Chaining this inequality over $k = 1, \dots, T$ and over sweeps $s = 0, \dots, S-1$ gives the final statement. □

B Method Details: Sequential Scheduling

This appendix details the scheduling of the per-task updates, summarized in Section 3.2.

A secondary design choice concerns how the per-task updates are scheduled within a refinement sweep. An aggregated expert-guided update would compute all u_i at $\theta^{(s)}$ and then apply $\eta \sum_i u_i$ at once; we instead apply each task's update immediately after computing it. This has two motivations. First, each update is applied at the same local model state on which its gradient, expert direction,

and gate were computed. Second, the expert-distance cap in Equation (4) controls each single-task movement before the next task sees the modified model, rather than allowing several task vectors to accumulate into one larger coordinate step.

Because the clip is applied *after* the learning-rate scaling, the per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η; a single update can never overshoot the expert in a coordinate no matter how large η is, and the learning rate controls only how many coordinates are driven to the cap. The sequential rule still does not guarantee Pareto improvement across tasks, because an update that helps task i can hurt task j; we therefore treat task ordering, cross-task interference, and worst-task retention as explicit diagnostics rather than as consequences of the gate alone. In practice, one can also normalize $u_i^{(s,k)}$ by tensor-wise RMS before applying the single-task update.

C Extended Experimental Protocol

This appendix expands the protocol of Section 4: benchmark positioning, secondary tracks, replay sampling, and the full baseline roster.

Benchmark settings. The multi-head GLUE-8 benchmark (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, excluding WNLI as is common) (Wang et al., 2018) uses RoBERTa-base as the encoder backbone (Liu et al., 2019): the shared encoder is merged and evaluated with each task's own trained head. This multi-head setup is convenient and common, but it assumes oracle task identity at evaluation time and should not be the only evidence for the method. The recent merging literature evaluates on three benchmark families—CLIP/ViT vision task-vector suites, GLUE and text-to-text NLP, and decoder-only LLM merging, the last standardized by MergeBench with released full-parameter Llama/Gemma experts (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024; Wang et al., 2019; Yu et al., 2023; He et al., 2025)—and we position the proposal against the first two; the decoder-only family is deferred until the variance of its generation-based evaluations is tightly controlled.

Secondary benchmark settings. Two secondary tracks address the limitations of multi-head GLUE. A CLIP/ViT vision track evaluates the standard task-vector image-classification suite—SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, DTD—testing a different modality, different loss geometry, and stronger domain shifts (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024), together with its twenty-task extension (Wang et al., 2024) (adding CIFAR-10/100, STL10, Flowers102, Oxford Pets, PCAM, FER2013, EMNIST, Food101, FashionMNIST, KMNIST, RenderedSST2), which probes the regime in which merging degrades most: with twenty task vectors competing for the same encoder weights, cross-task conflicts dominate and each vector must be down-weighted heavily to coexist (results in Section 5.1).

Replay data. For refinement, we use $n \in \{8, 16\}$ replay examples per task in the main grids (the twenty-task budget study extends to $n = 32$; Appendix E), sampled from training data or from a held-out split separated from the final validation split. Sampling is deterministic given the buffer seed. For classification tasks the draw is class-balanced: each class contributes $\lfloor n/C \rfloor$ examples and the remainder is filled uniformly at random. When a task has more classes than the budget holds—Cars (196) and SUN397 (397) at every budget we use, and DTD, RESISC45 and GTSRB at $n = 32$—the buffer cannot represent every class, so n classes are drawn at random per buffer seed and contribute one example each. Coverage is then part of what varies across draws, and fine-grained tasks are the ones for which a small buffer is least representative. STS-B, a regression task, is sampled uniformly. The replay and selection buffers are drawn from a single index stream and split disjointly, so they can never overlap. Sensitivity to the replay-budget size and to the buffer draw is reported for each track, rather than only the best tuned replay size.

Protocol details. The selection objective is the mean over tasks of the held-out per-task metric (Section 4). On a small fold the metric is quantized (one example in $8 \cdot B/2$) and cells can tie exactly; a tie goes to the least-moving candidate—the smaller η for the refinement arms, the smaller scaling coefficient and then the smaller density for a merge. On the fresh draws such ties occurred in 9 of 54 refinement cells, none of them on GLUE-8, and in 2 of 80 merge selections. Appendix C.1 records why the metric replaced the loss. Select-then-refit chooses η at construction size $B/2$ and

applies it at B; we accept this standard caveat rather than spend half the budget only on selection, and Appendix C.1 measures what it costs at $B = 16$. A selected cell counts as interior when both neighbouring grid values are worse; when it lies on a boundary the grid is extended in that direction and selection repeated (at most twice). Refinement runs S sweeps at a constant learning rate with the task order re-randomized every sweep from a fixed seed, so the D draws vary only the buffer; the expert-distance cap is applied per update (Eq. (4)). The horizon is fixed at $S = 50$ sweeps for every refinement family (Appendix C.1), and each family receives an equal-size η grid: $\{1, \ldots, 32\}$ (APR), $\{0.5, \ldots, 16\}$ (ungated), $\{10^{-5}, \ldots, 10^{-2}\}$ (GD) on the encoder suites, in factor-of-two steps, extended in the direction of a boundary selection. Buffer seeds 103–105 produce every reported draw; seeds 100–102 were used only to settle the protocol. AdaMerging uses the matched budget (the same B unlabeled inputs per task) rather than its transductive formulation on the evaluation split; DOGE/APGD builds its candidates without labels and uses the budget only to select its global scale. This accounting differs from common practice, where merge coefficients are tuned on the tasks' full validation splits outside any stated budget (Ilharco et al., 2022; Yadav et al., 2023) and that tuning dominates the cost of merging (He et al., 2025); here it happens inside the same B labels every method is charged for.

C.1 The selection rule, and the two ways it failed

The protocol of Section 4 selects on the tasks' own held-out metrics at a fixed horizon. It was not the first rule we ran, and because it was changed after seeing results we record the evidence for the change and the draws on which it was made. Buffer seeds 100–102 were used for every diagnosis below; every table in the body uses seeds 103–105, drawn afterwards. One simplification was made after the fresh draws had been run—the loss tie-break was dropped in favour of the least-moving cell—and the paragraph on ties below lists what it changed.

Held-out loss selects no-ops on GLUE-8. The first rule selected (η, S) over $S \in \{5, 20, 50, 100\}$ by the mean normalized held-out replay loss. On CLIP-8 it selects $\eta = 32$ on every draw, the cell the metric rule also selects. On GLUE-8 it selected a near-no-op ($\eta \leq 4$, displacement < 0.5 from θ_0) on roughly half of all draws, giving APR 0.52 ± 0.11 at $B = 32$ and 0.44 ± 0.09 at $B = 16$. The cause is the heads: RoBERTa's pretrained encoder predicts near-uniformly under each task's trained head, so the initial held-out loss is $\approx \ln C$ on every GLUE-8 task, a normalized loss of ≈ 1 is available to any model that does not move, and models that move far enough to score well raise the loss on some tasks before their metrics improve. Pinning the high-displacement cells the rule had rejected ($\eta \in \{8, 16, 32\}$, $S = 50$) scores 0.55–0.58 on the same draws; exchanging the roles of the two folds reproduces the same selections, so the failure belongs to the objective and not to a fold. Applied after the fact to the fresh draws, loss selection would again have chosen $\eta \in \{0.5, \ldots, 8\}$ on six of six GLUE-8 cells.

A two-dimensional metric grid ties. Replacing the loss by the held-out metric while keeping the (η, S) grid produced ties: on $B/2$ held-out examples per task the mean metric moves in steps of one example in $8 \cdot B/2$, and cells at different horizons reach the same value, so the tie-break was carrying a decision worth up to 0.1 on GLUE-8 ($S \leq 20$ against $S = 50$) while $S = 5$ collapses at $\eta = 32$ on CLIP-8. Fixing $S = 50$, the horizon every well-behaved draw had selected, removes the axis along which the ties formed, and η is then selected on the metric alone. While the rule was being settled we broke the remaining ties by the normalized loss; on the diagnostic draws that reproduces every CLIP-8 selection of the loss rule and reduces GLUE-8's spread from ± 0.11 to ± 0.02. The reported rule drops the loss entirely and gives a tie to the least-moving cell, which is the convention a reader would expect and keeps the loss out of the protocol in every role. On the fresh draws the two conventions differ in nine refinement cells—none on GLUE-8, and seven of them ablation cells whose tied rates are all near no-ops—and in two merge selections; the affected draws were regenerated under the least-moving rule, and the tables report those.

Select-then-refit at $B = 16$ on CLIP-8. The rule's one measured cost is the CLIP-8 $B = 16$ cell of Table 1. Cells are built on $B/2 = 8$ examples per task and the winner is refit on 16. On eight examples $\eta = 32$ genuinely collapses on some draws—held-out accuracy 0.34 with a normalized loss above 1 on one fresh draw, Cars at zero for $\eta = 64$ on another—so selection is right to reject it for the model it can see; on sixteen it is safe. Pinning $\eta = 32$ with the full refit on the two fresh draws that

rejected it scores 0.725 and 0.732 against their selected 0.675 and 0.634, and pinning $\eta = 8$ on the draw that selected 32 scores 0.677 against 0.733. Selection is therefore systematically conservative about η at this budget, and here the held-out loss would have chosen better (0.717 ± 0.022 against 0.681 ± 0.050, both below TIES's 0.734): the two suites prefer different objectives, and the unified rule pays on this cell for the protection it buys on GLUE-8.

Normalized retention. Earlier versions of this study aggregated GLUE-8 by normalized retention,

$$\mathrm{NormRet}_t = \frac{\mathrm{Score}_t(\theta_{\mathrm{merge}}) - \mathrm{Score}_t(\theta_0)}{\mathrm{Score}_t(\theta_t) - \mathrm{Score}_t(\theta_0) + \epsilon_{\mathrm{metric}}}, \tag{6}$$

which places the pretrained model at 0 and the task expert at 1 ($\epsilon_{\mathrm{metric}}$ guards small denominators). We keep it for the per-task appendix tables but no longer use it as an aggregate, because its denominators are the trouble: on MRPC the expert improves on the pretrained model by only 0.168, so one point of accuracy there is worth six of retention and that task alone dominates worst-task columns, and on the twenty-task CLIP suite several experts barely improve on the zero-shot backbone (STL10 by 0.004), so a near-zero denominator would make per-task retention explode. The plain mean of per-task metrics that the body reports weights tasks by their own scales, which is the convention of the GLUE benchmark itself.

Per-task metrics. CoLA is scored by Matthews correlation, STS-B by Pearson/Spearman correlation, MNLI by matched/mismatched accuracy, MRPC and QQP by accuracy/F1, and the remaining GLUE tasks by accuracy; every CLIP task is scored by top-1 accuracy.

Baselines. Because the proposed method uses labeled replay buffers, baselines are grouped by the data that constructs them—hyperparameter selection is common to every method and uses the labeled selection buffer (Section 4): *checkpoint-only construction* (pretrained encoder, single-task experts, uniform averaging, task arithmetic, TIES, DARE, DELLA-style sampling, Model Breadcrumbs, magnitude-based Localize-and-Stitch, DOGE/APGD); *unlabeled construction* (RegMean, AdaMerging, APL); *labeled construction* (ordinary replay gradient descent from the same initialization, distance-scaled descent without the gate, Fisher merging with an empirical diagonal Fisher estimated from per-example labeled-loss gradients on the replay buffer, and Localize-and-Stitch with masks learned on the replay buffer); and *sanity controls* (inverted gate, density-matched random gate, wrong-expert directions, shuffled labels). All labeled-replay baselines use the same replay examples, the same number of gradient evaluations where possible, and the same hyperparameter search budget.

For DOGE/APGD we port the authors' released `DOGE_TA` implementation: two-dimensional encoder weights only; per-layer $\lambda_i^l = \eta / \|\tau_i^l\|$; task and shared SVD ranks $\min(d_l)/T$ and $\min(d_l)/6$; 400 Adam iterations at learning rate 10^{-4} with gradients projected orthogonally to the shared basis; and the released inclusive global top-30% task-vector mask applied to each adjusted vector. We sweep $\eta \in \{0.075, 0.10, 0.15, 0.20\}$ on GLUE-8, $\{0.03, 0.05, 0.07, 0.09, 0.11, 0.13, 0.15, 0.18\}$ on CLIP-8, and $\{0.0025, 0.005, 0.0075, 0.01, 0.015, 0.02, 0.025, 0.03, 0.05\}$ on CLIP-20. The last grid is an audited extension after the initial $\{0.03, \ldots, 0.18\}$ grid hit its lower boundary; the boundary run was stopped before evaluation, and only the winner of the expanded grid was evaluated.

D Per-Task Results at the 8+8 Budget

Tables 7 and 8 give the task-level scores underlying the GLUE-8 and CLIP-8 columns of Table 4. Each row is the cell selected using the disjoint eight-example selection buffer; the evaluation scores below did not participate in selection. For GLUE-8, "score" denotes the benchmark metric used in the aggregate: Matthews correlation for CoLA, correlation for STS-B, and accuracy or accuracy/F1 for the remaining tasks. CLIP-8 reports top-1 accuracy.

E Twenty-Task CLIP: Extended Results

This appendix gives the full twenty-task campaign behind Section 5.1.

Table 7: GLUE-8 at the split-selected 8+8 budget: raw per-task evaluation scores for every nonempty GLUE-8 row of Table 4. The single-task-expert row evaluates each task's own expert and is included as a ceiling reference.

Initialization	Treatment	CoLA	SST-2	MRPC	STS-B	QQP	MNLI	QNLI	RTE
Pretrained	alone	0.000	0.509	0.748	−0.047	0.316	0.336	0.495	0.473
Single-task expert	alone	0.623	0.936	0.916	0.909	0.901	0.873	0.924	0.798
Pretrained	+GD	0.000	0.509	0.158	0.107	0.473	0.355	0.495	0.527
	+ungated	0.000	0.592	0.695	0.169	0.566	0.343	0.489	0.531
	+APR	0.033	0.787	0.692	0.741	0.572	0.397	0.532	0.458
Task arithmetic	alone	0.130	0.845	0.719	0.529	0.371	0.698	0.648	0.477
	+GD	0.057	0.888	0.755	0.715	0.812	0.715	0.793	0.531
	+ungated	0.017	0.878	0.775	0.821	0.792	0.747	0.738	0.513
	+APR	0.079	0.869	0.718	0.763	0.812	0.742	0.732	0.534
Breadcrumbs	alone	0.142	0.894	0.388	0.423	0.336	0.665	0.496	0.509
	+GD	−0.003	0.883	0.759	0.777	0.794	0.710	0.735	0.545
	+ungated	0.031	0.883	0.775	0.808	0.788	0.696	0.743	0.556
	+APR	0.065	0.867	0.739	0.810	0.803	0.717	0.763	0.563
DARE-TIES	alone	0.268	0.886	0.690	0.681	0.366	0.571	0.591	0.451
	+GD	0.194	0.896	0.789	0.822	0.639	0.611	0.714	0.448
	+ungated	0.129	0.893	0.762	0.809	0.780	0.655	0.772	0.523
	+APR	0.149	0.881	0.794	0.825	0.789	0.679	0.785	0.534
TIES	alone	0.195	0.862	0.712	0.286	0.330	0.461	0.570	0.458
	+GD	0.146	0.870	0.762	0.802	0.715	0.534	0.725	0.440
	+ungated	0.170	0.860	0.740	0.768	0.715	0.555	0.761	0.451
	+APR	0.190	0.860	0.767	0.795	0.748	0.581	0.745	0.477
AdaMerging	alone	−0.018	0.532	0.243	0.770	0.307	0.774	0.882	0.686
	+GD	−0.049	0.577	0.253	0.682	0.312	0.758	0.874	0.661
	+ungated	−0.026	0.588	0.731	0.815	0.750	0.756	0.883	0.588
	+APR	0.010	0.751	0.758	0.789	0.773	0.797	0.835	0.603

Table 8: CLIP-8 at the split-selected 8+8 budget: per-task top-1 accuracy for every nonempty CLIP-8 row of Table 4. The single-task-expert row evaluates each task's own expert and is included as a ceiling reference.

Initialization	Treatment	SUN397	Cars	RESISC	EuroSAT	SVHN	GTSRB	MNIST	DTD
Zero-shot	alone	0.632	0.597	0.582	0.450	0.315	0.325	0.482	0.439
Single-task expert	alone	0.749	0.783	0.949	0.991	0.972	0.989	0.996	0.794
Zero-shot	+GD	0.575	0.403	0.527	0.626	0.431	0.326	0.557	0.385
	+ungated	0.601	0.521	0.556	0.643	0.471	0.356	0.565	0.400
	+APR	0.617	0.512	0.640	0.818	0.776	0.662	0.928	0.447
Task arithmetic	alone	0.570	0.553	0.628	0.734	0.778	0.685	0.961	0.472
	+GD	0.553	0.391	0.692	0.868	0.856	0.691	0.753	0.487
	+ungated	0.555	0.450	0.659	0.847	0.830	0.748	0.947	0.477
	+APR	0.618	0.598	0.716	0.856	0.880	0.781	0.959	0.517
Breadcrumbs	alone	0.641	0.613	0.694	0.791	0.769	0.668	0.949	0.521
	+GD	0.637	0.605	0.734	0.862	0.836	0.749	0.946	0.522
	+ungated	0.617	0.521	0.735	0.860	0.826	0.747	0.940	0.502
	+APR	0.619	0.511	0.652	0.850	0.863	0.685	0.930	0.477
DARE-TIES	alone	0.594	0.577	0.663	0.713	0.862	0.792	0.977	0.486
	+GD	0.063	0.011	0.275	0.496	0.239	0.092	0.291	0.232
	+ungated	0.392	0.018	0.301	0.517	0.738	0.472	0.841	0.303
	+APR	0.646	0.538	0.773	0.867	0.902	0.860	0.971	0.522
TIES	alone	0.667	0.636	0.732	0.738	0.859	0.800	0.969	0.528
	+GD	0.666	0.630	0.779	0.876	0.898	0.848	0.960	0.539
	+ungated	0.655	0.305	0.762	0.828	0.877	0.820	0.950	0.514
	+APR	0.676	0.581	0.787	0.866	0.891	0.844	0.962	0.549
AdaMerging	alone	0.627	0.652	0.661	0.849	0.834	0.760	0.977	0.469
	+GD	0.624	0.645	0.710	0.896	0.877	0.813	0.965	0.481
	+ungated	0.620	0.334	0.716	0.904	0.866	0.804	0.965	0.469
	+APR	0.649	0.655	0.731	0.899	0.875	0.814	0.972	0.504

Held-out retention protocol. For Table 6, models are built on the standard eight-task CLIP suite and evaluated zero-shot on the twelve additional tasks in the twenty-task suite. The primary held-out score averages CIFAR-10, CIFAR-100, STL10, Flowers102, Oxford Pets, Food101, and FashionMNIST, the tasks on which the pretrained model performs meaningfully above chance; PCAM, FER2013, EMNIST, KMNIST, and RenderedSST2 are excluded from that average because their pretrained accuracies are at or near chance. Every data-dependent method receives the same eight inputs per training task, and each refinement uses the split-selected (η, S) from its CLIP-8 experiment. Only refinements from the pretrained model are reported, avoiding retention differences inherited from a merge initialization. For reference, transductive AdaMerging uses all $87{,}433$ evaluation inputs and reaches 0.759 training accuracy with a -0.031 held-out drop; it is excluded from the matched-budget table.

The vision track scales to the twenty-task suite of Wang et al. (2024) using the published fine-tuned ViT-B/32 encoders for all twenty tasks (every expert was validated to beat its zero-shot floor by a wide margin, including the three tasks whose class-name orderings we had to reconstruct: PCAM $0.556 \to 0.887$, FER2013 $0.376 \to 0.711$, RenderedSST2 $0.586 \to 0.713$).

Two protocol corrections forced by the scale. First, the uniform $\lambda_i = 0.3$ that is standard at eight tasks is catastrophic at twenty: the task-arithmetic merge scores 0.364 mean top-1 accuracy, *below* the 0.550 zero-shot floor, and the tuned optimum falls to $\lambda \approx 0.05$–0.08 (accuracy 0.604; the curve is monotone over $0.3 \to 0.08$ and flat below). Merge coefficients must be re-tuned per task count, and refinement must never start from a hand-picked global λ. Second, normalized retention (Eq. (6)) becomes *degenerate* at this scale: the suite contains tasks whose expert barely improves on the zero-shot backbone (STL10 gap 0.0043, Food101 gap 0.035), so the denominator explodes—STL10 alone produces per-task retentions near -60 and is the "worst task" of nearly every method for that reason only. On this suite we therefore report *absolute* mean and worst-task top-1 accuracy (zero-shot floor 0.550, expert ceiling 0.896) as primary, with retention restricted to non-degenerate tasks as a secondary check.

Why RegMean is an awkward anchor. RegMean is the initialization at which the anchored update is least advantaged, and the reason is specific to how RegMean sits in parameter space. RegMean solves each linear layer by matching *outputs* on that layer's inputs, $W = (\sum_i G_i)^{-1} \sum_i G_i W_i$; in input directions with negligible activation variance the solution is essentially unconstrained, so W acquires large components that are functionally inert on in-distribution inputs. The resulting merge point is over 450 from the base model in parameter norm—larger than the encoder norm itself (336)—while scoring a healthy 0.723, which is only possible because most of that displacement lies in inference-invisible directions. Every ingredient of APR reads the anchor $v_i = \theta_i - \theta$, so from this initialization v_i inherits that inflated, non-functional component: the step $-g \odot |v|$ scales noise on directions where $|v|$ is huge but the gradient is near zero, and the cap $|v|$ is widest exactly there—which is also why the usable rate collapses by an order of magnitude here. This yields a concrete prediction: anchoring on the functional part of v (equivalently, refining RegMean from a task-vector-defined anchor) should remove the handicap—an experiment we leave to future work. It also implies that parameter-space distance from base is a valid drift proxy for the task-vector-based methods but not for RegMean, whose functional drift must be measured in output space.

Where small buffers start to overfit. A methodological point matters here: best-per-cell tables cannot detect overfitting, because selected configurations exclude it by construction, so we audit *every* (η, S, n) cell for the overfitting signature (replay loss improving from $S = 30$ to $S = 60$ while test accuracy falls). For unanchored GD the signature appears as soon as the buffer is small: 9 of 36 small-n cells deteriorate, with textbook cases at $n = 16$ (replay $0.083 \to 0.063$ while test falls $0.636 \to 0.288$; a second cell falls $0.614 \to 0.540$), plus two outright instabilities where replay worsens as well. A practical corollary is that GD's safe learning-rate window narrows as sweeps accumulate—a rate tuned at $S = 5$ cannot be reused at $S = 60$. For the anchored update the signature never appears: across $\approx$48 cells the largest decline is 0.003, with no instabilities, *even though* APR also reaches near-interpolation (replay 0.007–0.011 on $n = 8$ buffers) and its optimal rate transfers unchanged across horizons. The absence of overfitting is therefore a property of the expert-distance cap, not of the data regime: once every coordinate is capped at $|v_{i,r}|$, additional sweeps have nowhere damaging to go, whereas GD keeps moving after it has fit the buffer.

Table 9: Replay-budget study on the twenty-task suite: best configuration over learning rate and horizon $S \in \{5, 30, 60\}$ per cell, mean top-1 accuracy (worst-task in parentheses); single seed, identical buffer draws across cells at each n. "Init. alone" is budget-independent. Ordinary GD is reported from the base model, the setting in which the anchored step's advantage is largest. Worst-task accuracy degrades faster than the mean as the buffer shrinks.

Initialization	Method	n=8	n=16	n=32
Base model (0.550 (0.100))	+ GD	0.573 (0.093)	0.580 (0.113)	0.602 (0.160)
	+ APR	**0.632** (0.187)	**0.640** (0.205)	**0.654** (0.236)
Task arith. (0.604 (0.115))	+ APR	0.650 (0.197)	0.659 (0.242)	0.674 (0.274)
TIES (0.626 (0.150))	+ APR	0.665 (0.224)	0.673 (0.250)	0.691 (0.284)
AdaMerging (0.665 (0.101))	+ APR	0.683 (0.207)	0.688 (0.249)	0.698 (0.241)

Geometry of the twenty-task regime. Direct measurement explains why the anchor's advantage concentrates on refinement from the pretrained model. At the tuned merge, every expert sits at distance ≈ 2.4 (encoder norm 336), but the best refined solutions found by *any* stable method lie only 0.14–0.23 from a strong initialization—about 8% of one expert distance. As tasks accumulate, the achievable joint optimum stays close to a good initialization (a compromise among twenty mutually incompatible experts, mirrored by the tuned λ falling from 0.2–0.3 to 0.08), while the expert distances $|v_i|$ that set APR's stride stay large; the two length scales that coincided at three and eight tasks decouple at twenty. This is the mechanism behind the shrinking gate advantage and the shrinking margin over GD from a merge, and it makes a testable prediction: from the *pretrained model*, where the destination genuinely is far, the distance-scaled step should dominate small-step GD again. It does—the margin is $\approx +0.05$–0.06 mean accuracy at every budget (Table 9), because the good region is genuinely far (> 1) while GD's small steps travel only 0.09–0.22 and cannot reach it.

Replay budget at twenty tasks. We run a full budget grid: $n \in \{8, 16, 32\}$ labeled examples per task, each refined at $S \in \{5, 30, 60\}$ sweeps with the learning rate tuned per cell, from all four starting points, with identical buffer draws across runs (single seed). Table 9 reports the best configuration over the learning rate *and* the horizon, since the two interact: at small n nearly every best cell uses $S \geq 30$—more passes genuinely compensate for fewer examples—and the optimal rate falls as the buffer shrinks (to $\eta \approx 1$–4), while extending further to $S = 60$ moves best configurations by at most 0.004 at any budget, so the horizon is effectively converged by thirty sweeps. Three regularities. First, degradation with budget is graceful in the mean but concentrated in the *worst task*: from AdaMerging, mean accuracy loses 0.015 across the $4\times$ budget reduction ($0.698 \to 0.683$) while worst-task accuracy loses more than twice as much in relative terms ($0.241 \to 0.207$)—the cost of a small buffer is paid disproportionately by the hardest task, which a twenty-task mean conceals. Second, the refinement's value survives at the smallest budget: with just *eight* labeled examples per task, refining the base model reaches 0.632 (0.626 ± 0.005 over five buffer draws, above the merge on every draw)—while the tuned task-arithmetic merge (0.604) uses no labels but requires all twenty expert checkpoints and a tuned coefficient. Third, from the base model the anchored update's margin over plain gradient descent is large and essentially independent of the budget ($\approx +0.05$–0.06 mean at every n; GD row in Table 9): the geometry argument above, not data volume, sets that gap. Over five pinned buffer draws at $n = 8$ that margin is $+0.053 \pm 0.006$ (APR 0.626 ± 0.005 against GD 0.572 ± 0.003), and APR's worst-task accuracy is higher on every draw (0.170 ± 0.017 against 0.095 ± 0.011).

Summary of the twenty-task study. Four claims survive the campaign, each with its evidence. (i) *The anchored refinement improves every initialization it is given*, including the pretrained model itself, with the gain shrinking as the initialization improves but never vanishing (Table 9, and the CLIP-20 column of Table 3). (ii) *The load-bearing ingredient is the anchored, distance-scaled, clipped step, not the attribution gate*: the gate is neutral on mean accuracy at this scale, and its thresholded variants cannot help, because the gradient's coordinate-level selection carries no transferable structure (the held-out audit above). (iii) *Safety is the method's clearest practical advantage*: across every learning rate, horizon, and budget tested, no anchored-refinement run fell below its initialization or the zero-shot floor, and none showed the overfitting signature, while unconstrained descent deteriorates on a quarter of its small-buffer cells. (iv) *The method works in the small-data regime*: eight labeled

examples per task suffice to beat the tuned task-arithmetic merge from the pretrained model alone, by $+0.053 \pm 0.006$ over ordinary descent across five buffer draws, with the budget's cost concentrated in worst-task accuracy.

Limitations. Four gaps remain. The budget grid of Table 9 is single-seed, with five-seed replication only at its from-pretrained $n = 8$ cell, and the cells of Table 3 are single-draw throughout, as are the held-out retention study of Table 6 and the drift probe of Table 5. On the twenty-task suite, the coefficient grid offered to split selection for task arithmetic ends at $\lambda = 0.2$, above that benchmark's optimum, so the selected merge is a boundary pick that lands below the zero-shot floor, and the from-task-arithmetic refinement rows of Table 3 inherit that starting point; a selection grid extended below $\lambda = 0.2$ would need those cells re-run. The alternative labeled baselines are re-run under split selection at the $16{+}16$ budget only (Table 3); the $8{+}8$ grids carry APR and its two ablations alone. The predicted functional-anchor fix for the RegMean initialization is untested. The held-out retention study is single-seed, at one budget, on one split, and that split is a comparatively low-interference regime—eight training tasks, where merging has less to repair and the strongest label-free merge is correspondingly hard to beat; whether the ordering against AdaMerging changes as interference grows is untested here. RegMean's retention is likewise budget-dependent: worst in the table at an eight-input Gram budget, it behaves quite differently at larger ones. A further limitation is one of scope rather than execution: the results reported here are confined to budgets of at most 32 labeled examples per task, so we make no claim about how the ordering between the anchored and unconstrained families behaves when labeled data is plentiful.